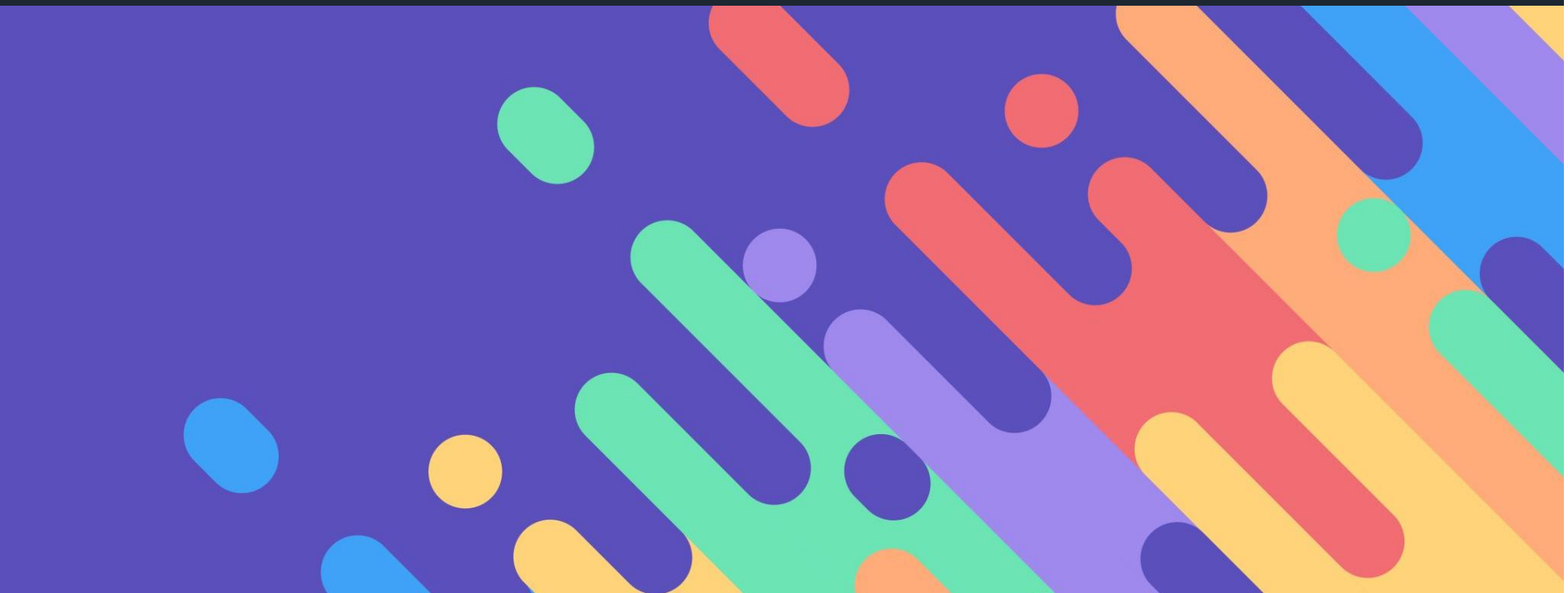
Generative AI: Large Language Models



DIGITAL LINGUISTICS AND LANGUAGE PROCESSING

Structure

- Fundamentals of large language models
  - How LLMs differ from LMs?
  - Datasets
  - Basics of text generation
  - Hardware requirements
  - Open-source vs. commercial LLMs
- Demo: Google colab with emphasis on Open-source LLMs

LMs vs. LLMs

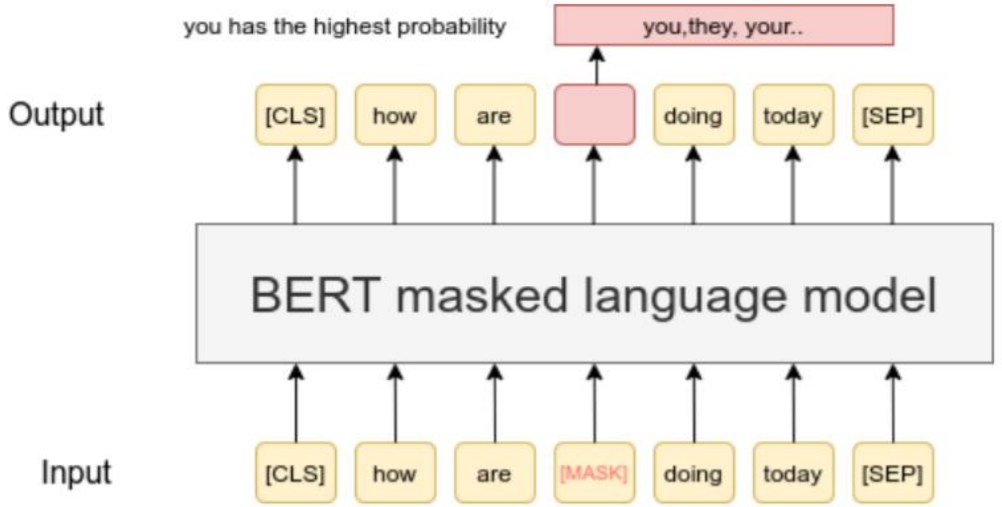
Feature	Language Models (LMs)	Large Language Models (LLMs)
Size	Smaller number of parameters	Billions of parameters
Training Data	Domain-specific or limited data	Vast, diverse datasets covering various topics
Context Handling	Limited context understanding	Strong context handling, long-range dependencies
Performance	Adequate for simple tasks	State-of-the-art performance across complex tasks
Scalability	Easier to train and deploy	Requires significant computational resources
Use Cases	Specific, narrow applications	General-purpose, capable of zero-shot learning

Applications of LMs vs. LLMs

- Language Models (LMs, smaller & easier tasks):
  - Text Completion: Basic sentence completion and text prediction.
  - Speech Recognition: Recognizing and transcribing spoken language.
  - Spell Checking: Detecting and correcting spelling errors.
- Large Language Models (LLMs, challenging NLP tasks):
  - Content Generation: Writing articles, stories, and creating dialogue.
  - Conversational AI: Building advanced chatbots and virtual assistants.
  - Translation: Providing high-quality, context-aware translations.
  - Summarization: Condensing large texts into concise summaries.
  - Question Answering: Answering complex questions with detailed responses.

Pre-training Models

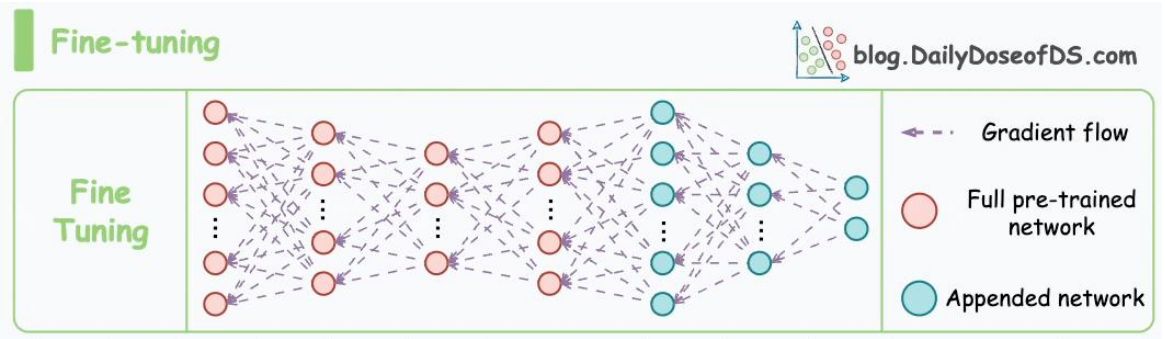
- **Self-supervised learning** involves learning from unlabeled data, using the data itself to create training objectives.
- **Next Token Prediction**: The model predicts the next word in a sequence based on the previous words.
- **Masked Language Modeling (MLM):** Certain words in a sentence are masked or hidden, and the model learns to predict these masked words using the surrounding context.
- **Result**: A general knowledge/understanding of language.



Source: https://raw.githubusercontent.com/UKPLab/sentence-transformers/master/docs/img/MLM.png

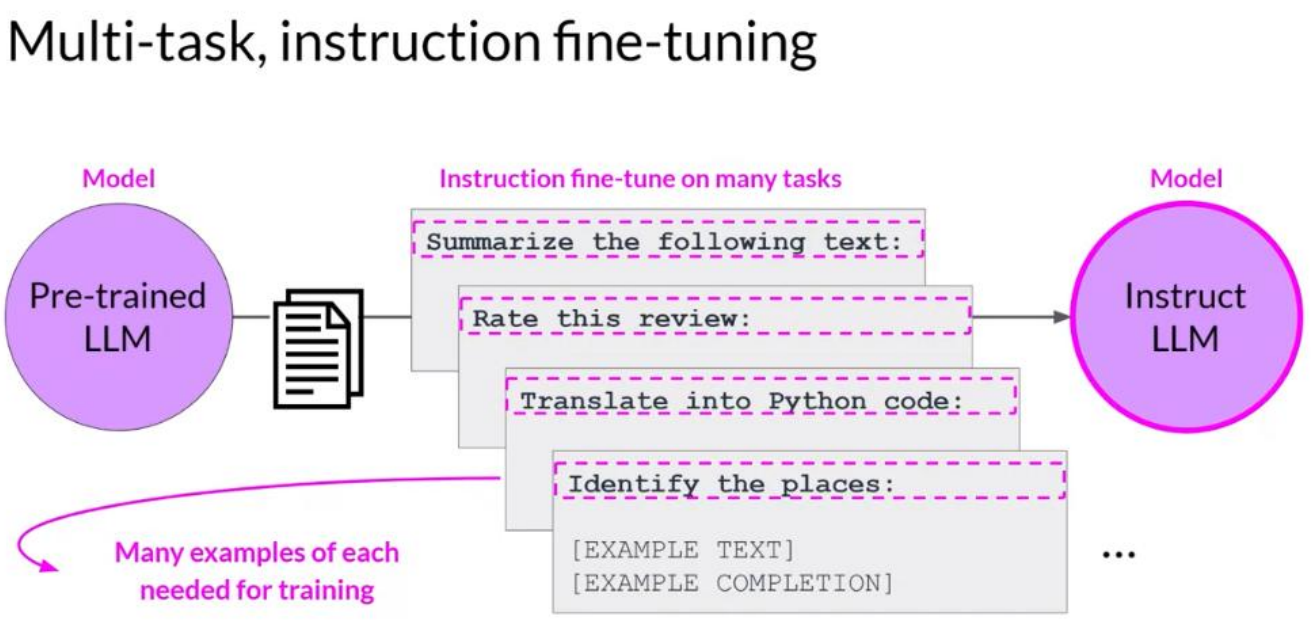
Fine-tuning models

- **Fine-tuning**: Adapting a pre-trained model to specific tasks by further training on a smaller, task-specific dataset.
- On top of LM, we attach e.g. classification head.
- Key Steps:
  1. **Start with a Pre-trained Model**: Utilize a general-purpose model trained on large datasets.
  2. **Prepare Task-Specific Data**: Collect relevant, labeled data for the specific task.
  3. **Fine-Tune the Model**: Train with a low learning rate to adjust model weights for the task.



Instruction-tuning Datasets

- **Instruction tuning**: Fine-tuning language models on a collection of datasets described via instructions.
- **Goal:** Designed to train LLMs to understand and follow natural language instructions.
- They often include human feedback (ranking).
- Alignment with ethical guidelines and user safety.

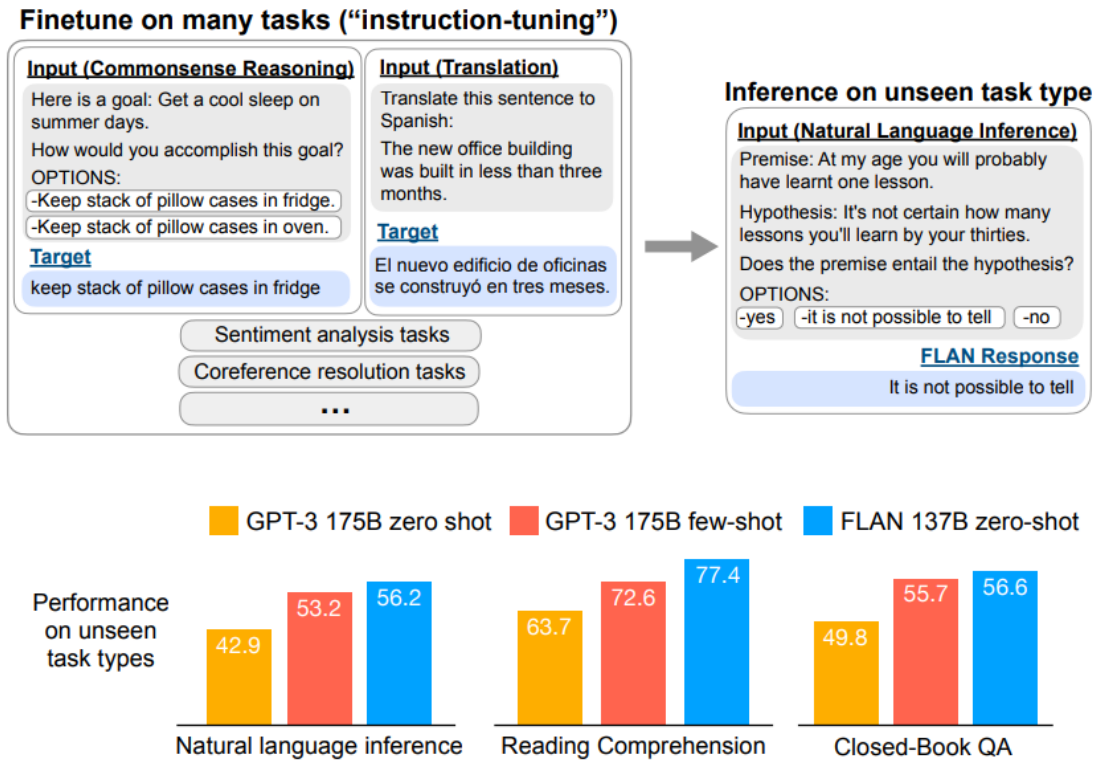


Source: https://medium.com/@yash9439/introduction-to-llms-and-the-generative-ai-part-3-fine-tuning-llm-with-instruction-and-326bc95e07ae

Instruction-tuning Datasets

- They substantially improve zero-shot performance on unseen tasks.
- Examples (2022)
  - **FLAN (Fine-tuned Language Net)**: *Finetuned Language Models are Zero-Shot Learners*
  - **T0 (T5 Zero-shot)**: *Multitask Prompted Training Enables Zero-Shot Task Generalization*
    - It is created by reformatting existing NLP datasets into a prompt-based structure

FINETUNED LANGUAGE MODELS ARE ZERO-SHOT LEARNERS



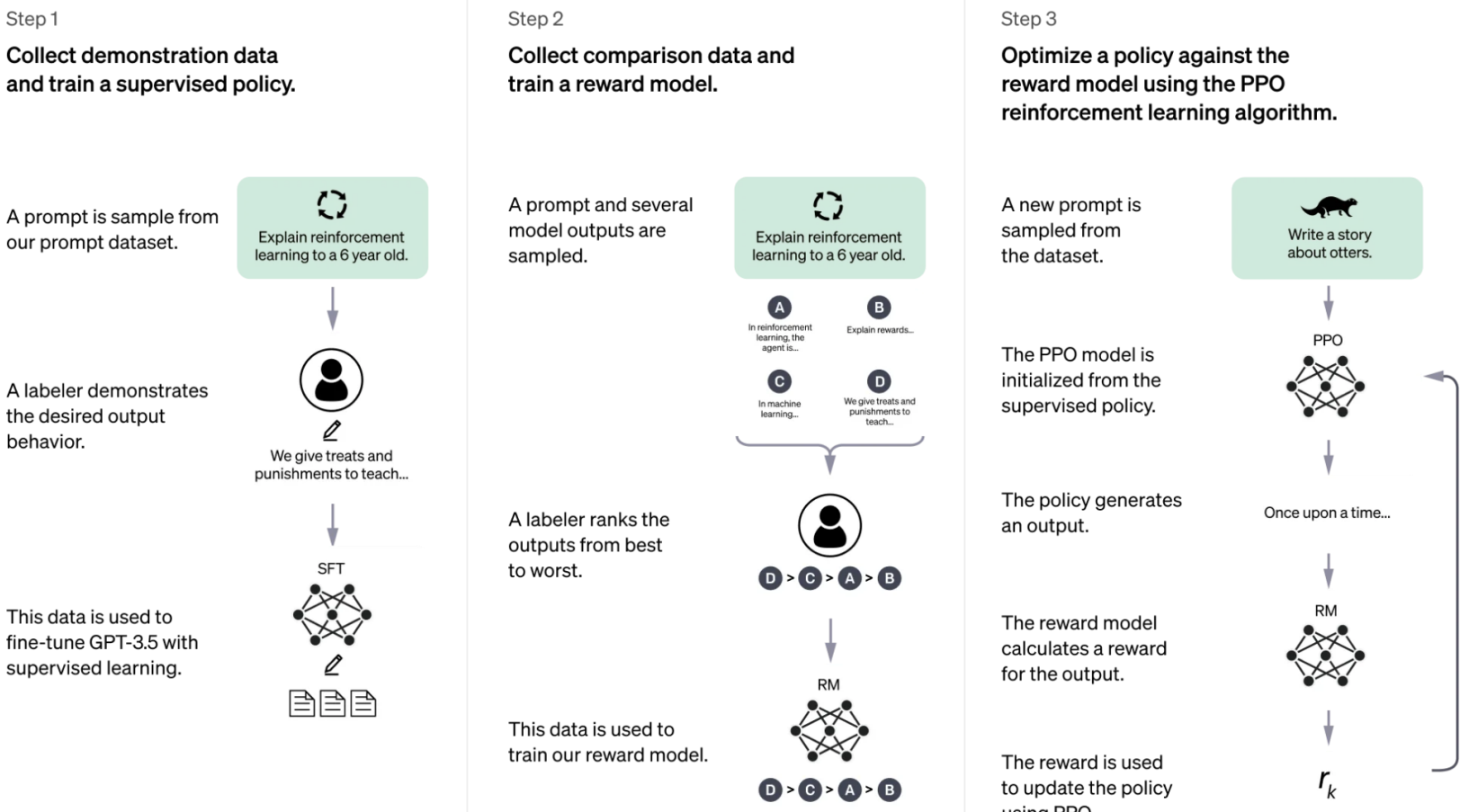
Source: https://arxiv.org/pdf/2109.01652

Model training types

	Pre-Training	Fine-Tuning	Instruction Tuning
Definition	Initial training of a model on a large, diverse dataset to learn general language patterns and knowledge.	Further training a pre-trained model on a task-specific or domain-specific dataset.	Specialized fine-tuning where the model learns to follow natural language instructions.
Goal	Equip the model with broad, foundational knowledge and language understanding.	Adapt the model to perform better on specific tasks or in specific domains.	Improve the model's ability to understand and follow natural language instructions.
Use Cases	Creating a general-purpose language model capable of understanding and generating text.	Sentiment analysis, translation, domain adaptation (e.g., legal, medical).	Task execution based on user instructions, enhancing versatility in handling diverse tasks.
Dataset Type	Large-scale, general datasets from various sources (e.g., internet text, books, articles).	Task-specific or domain-specific text data.	Pairs of natural language instructions and corresponding outputs.
Relation	Foundation for fine-tuning and instruction tuning, providing the initial language understanding.	Builds on pre-trained models to specialize in specific areas.	A specific form of fine-tuning targeting instruction adherence, building on both pre-training and fine-tuning.

OpenAI ChatGPT

- ChatGPT case
- Fine-tuned on GPT (Transformer decoder) & Optimized for chat
- Reinforcement-learning from human feedback (RLHF)



Generative AI

- What is Generative AI? Generative AI creates new content by learning patterns from existing data. It mimics human creativity to produce original text, images, music, and more.
- How it is done? ***By sampling from a learned distribution.***
- Examples of Generative Tasks
  - Text Generation: Creating written content like articles, stories, and code.
  - Image Synthesis: Producing or modifying images, including digital art and deepfakes.
  - Music Composition: Generating original music pieces or soundtracks.

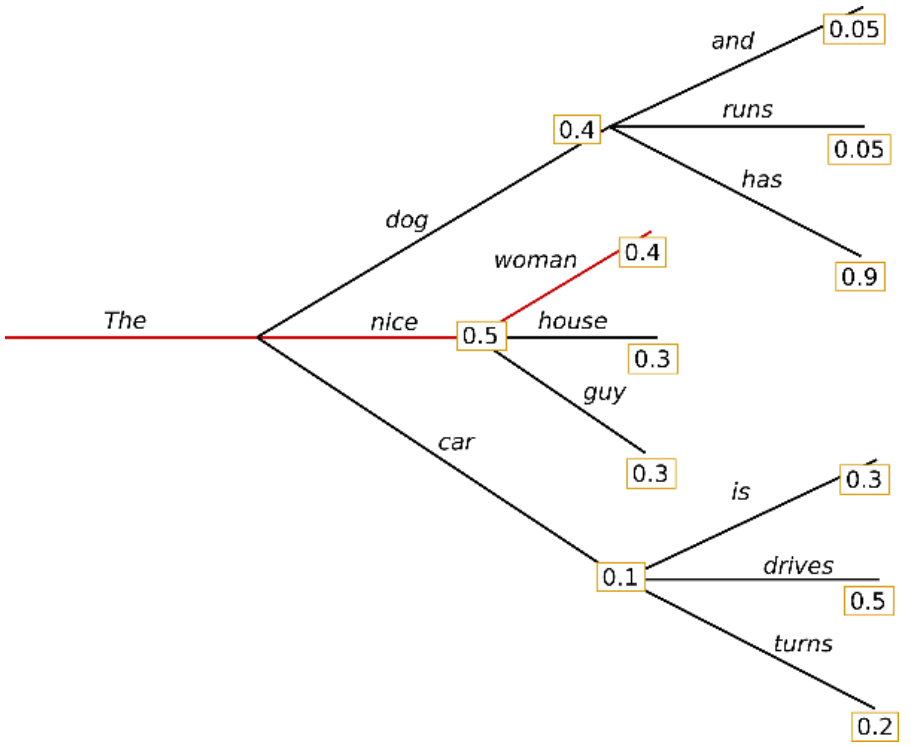
Source: DALL-E (OpenAI)

Text Generation

- What are Text Generation Models?
  - AI systems that produce human-like text
  - Applications: Chatbots, content creation, translation, code generation and more
- Popular variant is GPT (Generative Pre-trained Transformer)
  - GPT-3, GPT-4, etc., with increasing capabilities
- Text Generation Challenges
  - Ensuring coherence, relevance, and avoiding biases
  - Handling different languages and domains
- **Question: How to use LM output probabilities?**

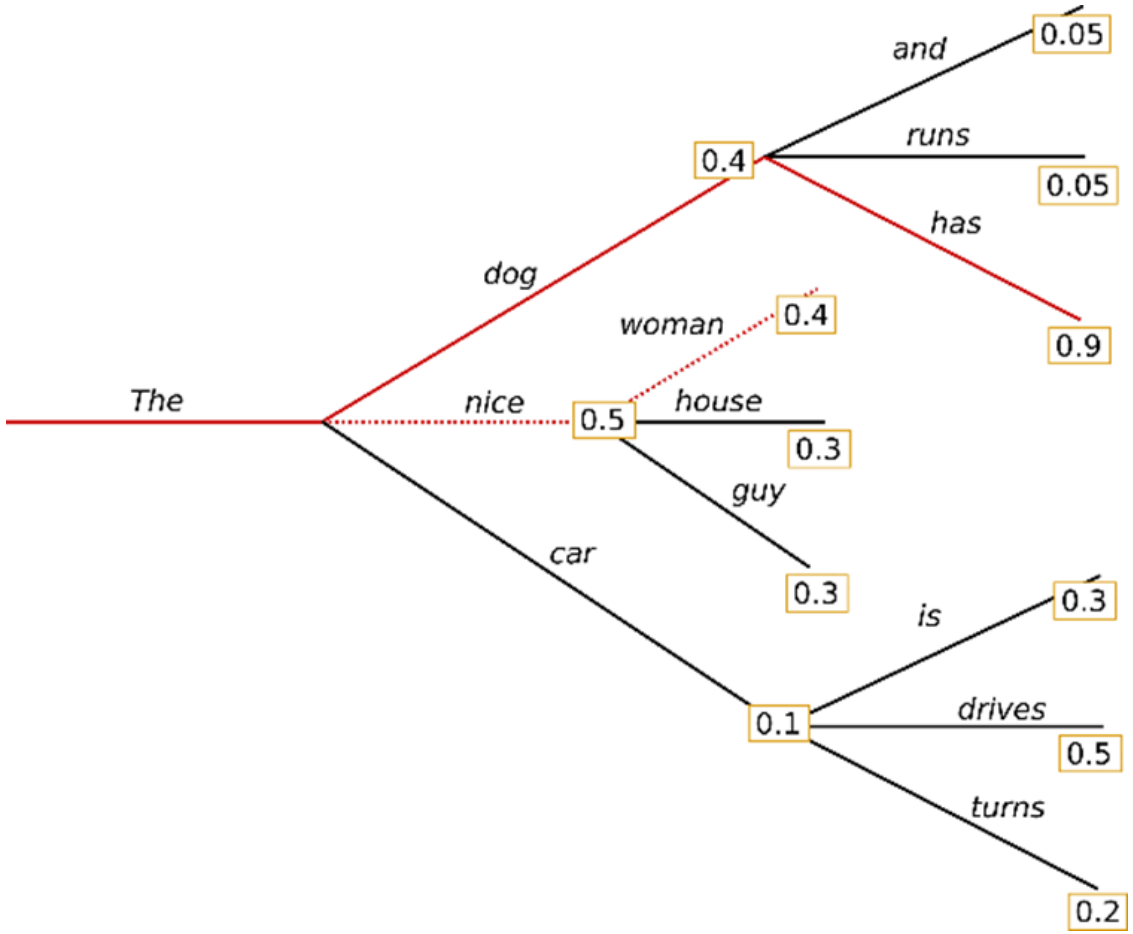
Decoding Strategies: Greedy Search

- **Greedy search**: the model selects the token with the highest probability as the next word in the sequence.
- Pros:
  - Fast and computationally efficient.
  - Easy to implement.
- Cons:
  - May produce repetitive or generic outputs.
  - It doesn't consider future possibilities or the overall sequence quality.



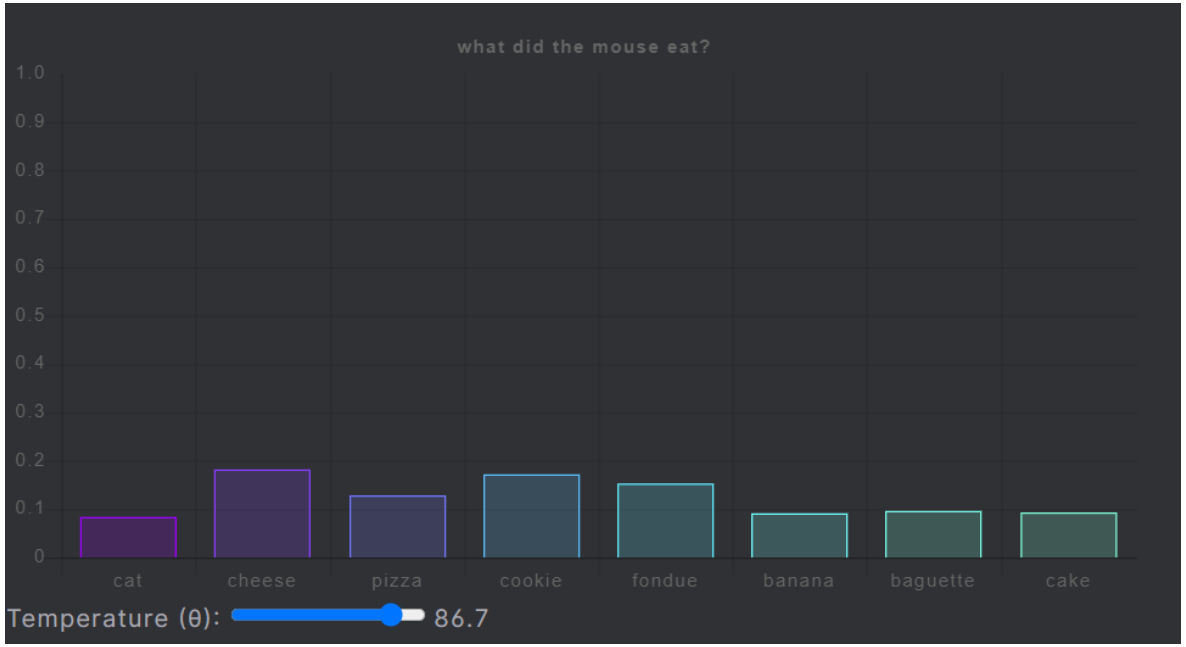
Decoding Strategies: Beam Search

- **Beam search:** Maintains multiple candidate sequences (or "beams") at each step.
- Pros:
  - Usually produces better text compared to greedy search.
  - Balances between exploration (considering different possibilities) and exploitation (selecting high-probability tokens).
- Cons:
  - More computationally intensive than greedy search.



Decoding Strategies: Sampling

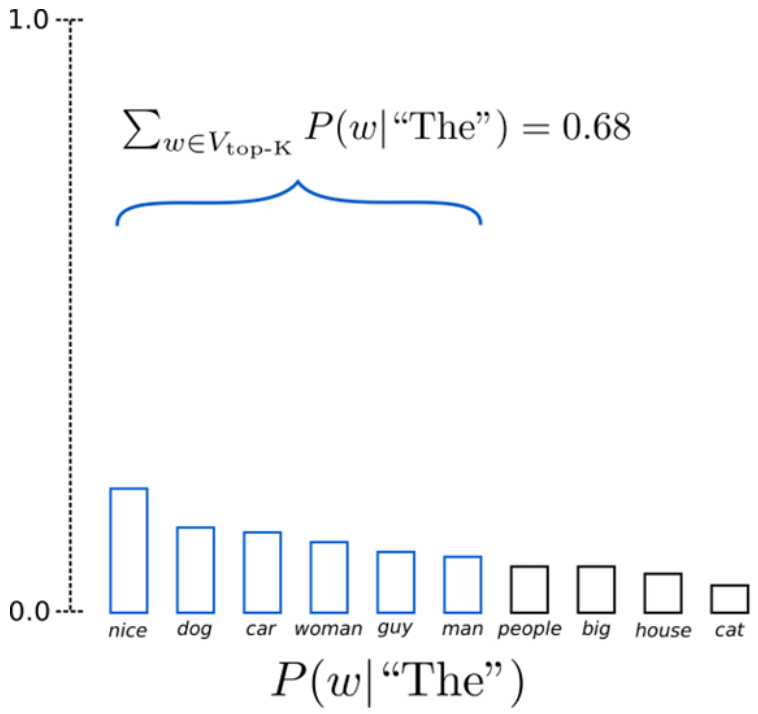
- **Sampling**: Decoding technique where the next word is randomly selected from the probability distribution output by the model, instead of always choosing the most likely word.
- This introduces ***variability*** and can lead to more diverse and creative outputs.
- **Temperature Sampling**: Adjusts the probability distribution to make the model more or less confident.
  - A higher temperature makes the distribution more uniform (more randomness), while a lower temperature makes it peakier (less randomness).

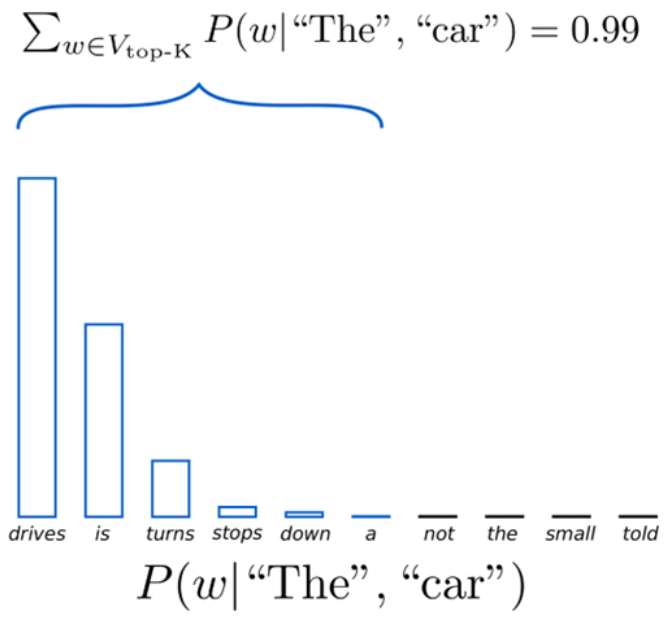


Source: https://lukesalamone.github.io/posts/what-is-temperature/

Top-k Sampling

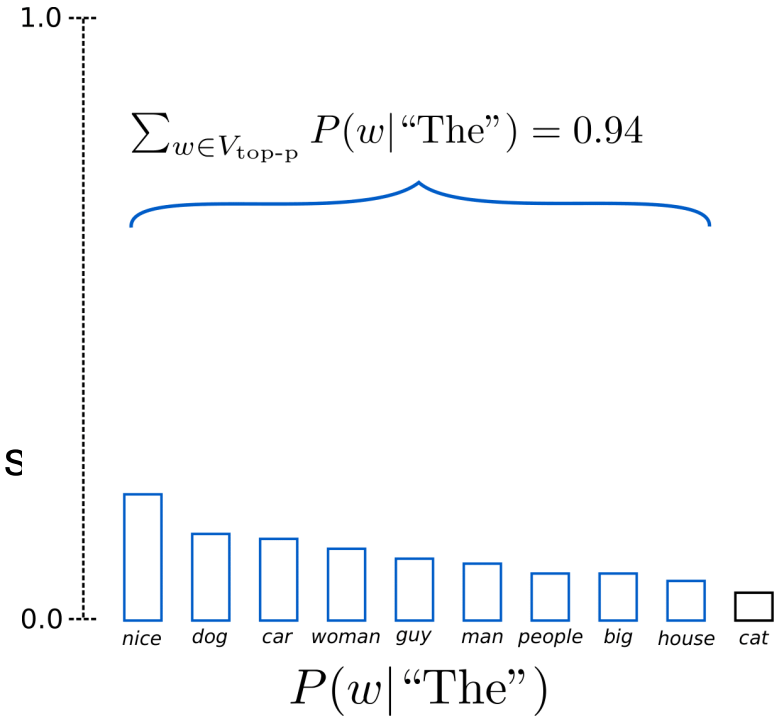
- ***Limits the selection to the top-k most probable words.***
- Ensuring that only the most likely options are considered.

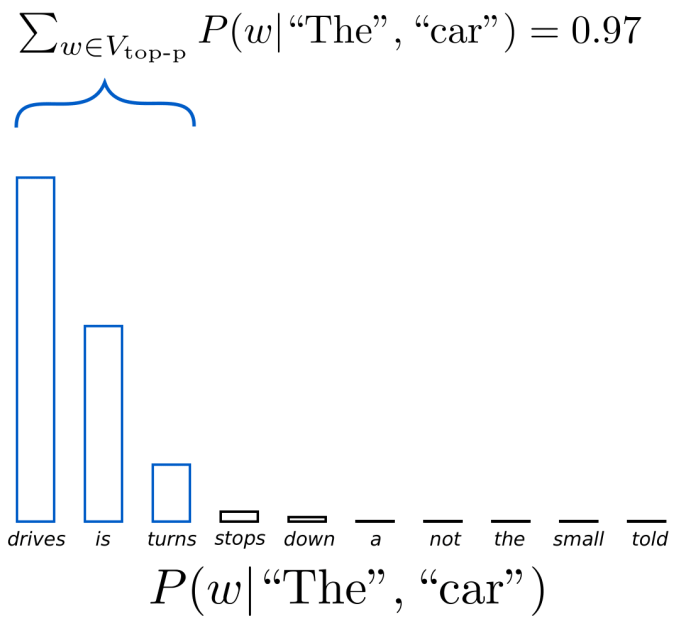




Top-p Sampling

- Top-p sampling selects the smallest set of words **whose cumulative probability exceeds a predefined threshold *p***.
- Top-p sampling *adaptively* chooses the top words that collectively account for at leas p percent of the probability mass.





Advanced: Contrastive search

- Contrastive search ***balances the trade-off between diversity and coherence.***
- *Model confidence*: the probability of candidate *v* predicted by the model.
- *Degeneration penalty*: discourages repetitive, uninformative, or bland outputs
- Hyperparameter *alpha* regulates the importance of these two components. *Alpha = 0* means greedy search. The variable *S* expresses cosine similarity between candidate and context.

$$x_t = \arg\max_{v \in V^{(k)}} \left\{ (1-\alpha) \times \underbrace{p_\theta(v|\boldsymbol{x}_{<t})}_{\text{model confidence}} - \alpha \times \underbrace{(\max\{s(h_v, h_{x_j}) : 1 \leq j \leq t-1\})}_{\text{degeneration penalty}} \right\}$$

Challenges of text generation

- **Consistency & Coherence**: Ensuring that the generated text is logically consistent and stays relevant to the given context or prompt.
- **Hallucinations**: Producing text that is factually correct and based on accurate information.
- **Bias**: Generating content that is free from bias, offensive language, or harmful stereotypes

OpenAI Parameters

- Better understanding of parameters
- Better understanding of LLMs

`temperature` number or null Optional Defaults to 1

What sampling temperature to use, between 0 and 2. Higher values like 0.8 will make the output more random, while lower values like 0.2 will make it more focused and deterministic.

We generally recommend altering this or `top_p` but not both.

`top_p` number or null Optional Defaults to 1

An alternative to sampling with temperature, called nucleus sampling, where the model considers the results of the tokens with top_p probability mass. So 0.1 means only the tokens comprising the top 10% probability mass are considered.

We generally recommend altering this or `temperature` but not both.

LLM Hardware Requirements

70B Model

Model: Meta-Llama-3-70B-Instruct

Number of parameters (in billions): 70

Precision: float16

Batch Size: 1

Sequence Length: 2048

Hidden Size: 8192

Number of Layers: 80

Number of Attention Heads: 64

Inference | Training

Total Inference Memory: 142.51 GB

- **Model Weights**: 130.39 GB
- **KV Cache**: 5.00 GB
- **Activation Memory**: 7.12 GB

Inference | Training

Total Training Memory: 924.82 GB

- **Model Weights**: 130.39 GB
- **KV Cache**: 5.00 GB
- **Activation Memory**: 7.12 GB
- **Optimizer Memory**: 521.54 GB
- **Gradients Memory**: 260.77 GB

8B Model

Model: Meta-Llama-3-8B-Instruct

Number of parameters (in billions): 8

Precision: float16

Batch Size: 1

Sequence Length: 2048

Hidden Size: 4096

Number of Layers: 32

Number of Attention Heads: 32

Inference | Training

Total Inference Memory: 19.46 GB

- **Model Weights**: 14.90 GB
- **KV Cache**: 1.00 GB
- **Activation Memory**: 3.56 GB

Inference | Training

Total Training Memory: 108.87 GB

- **Model Weights**: 14.90 GB
- **KV Cache**: 1.00 GB
- **Activation Memory**: 3.56 GB
- **Optimizer Memory**: 59.60 GB
- **Gradients Memory**: 29.80 GB

Quantization

- Quantization is a process of reducing the precision of the numbers (weights and activations) used in a model, typically from 32-bit floating point (FP32) to lower precision formats like 8-bit (INT8).
- **Why:** Reduced model size, faster inference, and lower power consumption

float16

Inference Training

Total Inference Memory: 19.46 GB

- **Model Weights**: 14.90 GB
- **KV Cache**: 1.00 GB
- **Activation Memory**: 3.56 GB

int8

Inference Training

Total Inference Memory: 11.51 GB

- **Model Weights**: 7.45 GB
- **KV Cache**: 512.00 MB
- **Activation Memory**: 3.56 GB

int4

Inference Training

Total Inference Memory: 7.54 GB

- **Model Weights**: 3.73 GB
- **KV Cache**: 256.00 MB
- **Activation Memory**: 3.56 GB

Quantization

- The reduction in performance due to quantization is **minimal**, making it a viable option.

Model	Datatype	Quantization Method	Average Accuracy
Qwen-7B-Chat	BFloat16	-	57.10
	INT-8	LLM.int8()	56.67
		GPTQ	57.21
		SpQR	56.49
	INT-4	GPTQ	54.86
		SpQR	56.41
	INT-3	GPTQ	51.42
		SpQR	55.45
	INT-2	GPTQ	16.52
		SpQR	53.18

Source: https://aclanthology.org/2024.findings-acl.726/

Open-source vs. Commercial LMs

- Open-Source LLMs
  - **Llama (Meta), Gemma (Google), Mistral (Mistral AI), Qwen (Alibaba Cloud)**
  - Freely available to the public.
  - Highly customizable: Users can fine-tune, modify, and adapt the model to specific needs.
  - Promotes community-driven development and innovation.
  - Transparency allows scrutiny for security vulnerabilities.
- Commercial LLMs
  - **GPT-4 (OpenAI), Gemini (Google), Claude (Antropic)**
  - Proprietary and often behind paywalls or APIs.
  - Access may require subscriptions, licensing fees, or agreements.
  - Limited access to the underlying model and codebase.
  - Data privacy concerns.
  - Open-science?

Performance Comparison

Category	Benchmark	Llama 3 8B	Gemma 2 9B	Mistral 7B	Llama 3 70B	Mixtral 8x22B	GPT 3.5 Turbo	Llama 3 405B	Nemotron 4 340B	GPT-4 (0125)	GPT-4o	Claude 3.5 Sonnet
General	MMLU (5-shot)	69.4	**72.3**	61.1	**83.6**	76.9	70.7	87.3	82.6	85.1	89.1	**89.9**
	MMLU (0-shot, CoT)	**73.0**	72.3△	60.5	**86.0**	79.9	69.8	88.6	78.7◁	85.4	**88.7**	88.3
	MMLU-Pro (5-shot, CoT)	**48.3**	–	36.9	**66.4**	56.3	49.2	73.3	62.7	64.8	74.0	**77.0**
	IFEval	**80.4**	73.6	57.6	**87.5**	72.7	69.9	**88.6**	85.1	84.3	85.6	88.0
Code	HumanEval (0-shot)	**72.6**	54.3	40.2	**80.5**	75.6	68.0	89.0	73.2	86.6	90.2	**92.0**
	MBPP EvalPlus (0-shot)	**72.8**	71.7	49.5	**86.0**	78.6	82.0	88.6	72.8	83.6	87.8	**90.5**
Math	GSM8K (8-shot, CoT)	**84.5**	76.7	53.2	**95.1**	88.2	81.6	**96.8**	92.3◇	94.2	96.1	96.4◇
	MATH (0-shot, CoT)	**51.9**	44.3	13.0	**68.0**	54.1	43.1	73.8	41.1	64.5	**76.6**	71.1
Reasoning	ARC Challenge (0-shot)	83.4	**87.6**	74.2	**94.8**	88.7	83.7	**96.9**	94.6	96.4	96.7	96.7
	GPQA (0-shot, CoT)	32.8	–	28.8	**46.7**	33.3	30.8	51.1	–	41.4	53.6	**59.4**
Tool use	BFCL	**76.1**	–	60.4	84.8	–	**85.9**	88.5	86.5	88.3	80.5	**90.2**
	Nexus	**38.5**	30.0	24.7	**56.7**	48.5	37.2	**58.7**	–	50.3	56.1	45.7
Long context	ZeroSCROLLS/QuALITY	81.0	–	–	90.5	–	–	**95.2**	–	**95.2**	90.5	90.5
	InfiniteBench/En.MC	65.1	–	–	78.2	–	–	**83.4**	–	72.1	82.5	–
	NIH/Multi-needle	98.8	–	–	97.5	–	–	98.1	–	**100.0**	**100.0**	90.8
Multilingual	MGSM (0-shot, CoT)	**68.9**	53.2	29.9	**86.9**	71.1	51.4	**91.6**	–	85.9	90.5	**91.6**

Source: https://arxiv.org/pdf/2407.21783

Evaluation

- Chatbot Arena: https://lmarena.ai/
- **Chatbot Arena is an open-source research project** developed by members from LMSYS and UC Berkeley SkyLab.
- How it works?
  - User asks any question to two anonymous LLMs
  - User votes for the better one

Rank* (UB)	Model	Arena Score	95% CI	Votes	Organization	License	Knowledge Cutoff
1	ChatGPT-4o-latest (2024-08-08)	1316	+4/-4	24023	OpenAI	Proprietary	2023/10
2	Gemini-1.5-Pro-Exp-0827	1301	+5/-5	19910	Google	Proprietary	2023/11
2	Gemini-1.5-Pro-Exp-0801	1298	+4/-4	25211	Google	Proprietary	2023/11
2	Grok-2-08-13	1295	+6/-6	10019	xAI	Proprietary	2024/3
5	GPT-4o-2024-05-13	1286	+3/-2	82934	OpenAI	Proprietary	2023/10
6	GPT-4o-mini-2024-07-18	1274	+4/-4	23147	OpenAI	Proprietary	2023/10
6	Gemini-1.5-Flash-Exp-0827	1271	+7/-6	6282	Google	Proprietary	2023/11
6	Claude 3.5 Sonnet	1270	+3/-3	53352	Anthropic	Proprietary	2024/4
6	Gemini Advanced App (2024-05-14)	1266	+3/-3	52225	Google	Proprietary	Online
6	Meta-Llama-3.1-405b-Instruct	1266	+3/-5	24584	Meta	Llama 3.1 Community	2023/12
6	Grok-2-Mini-08-13	1265	+6/-5	10791	xAI	Proprietary	2024/3

Source (September 3, 2024): https://lmarena.ai/

Ethical Considerations and Challenges

- **Bias and Fairness**: Address the issues of bias in training data and how it can lead to biased outputs.
- **Misinformation and Misuse**: Risks of generating misleading or harmful information.
- **Privacy Concerns**: Handling sensitive data and ensuring privacy.
- **Environmental Impact**: Discuss the computational resources required to train LLMs and their carbon footprint.

Useful links

- Prompt engineering: https://platform.openai.com/docs/guides/prompt-engineering
- Prompting guide: https://www.promptingguide.ai/techniques
- Huggingface (models): https://huggingface.co/models
- Huggingface (datasets): https://huggingface.co/datasets
- But what is a GPT (3Blue1Brown): https://www.youtube.com/watch?v=wjZofJX0v4M&t=3s
- Attention mechanism (3Blue1Brown): https://www.youtube.com/watch?v=eMlx5fFNoYc